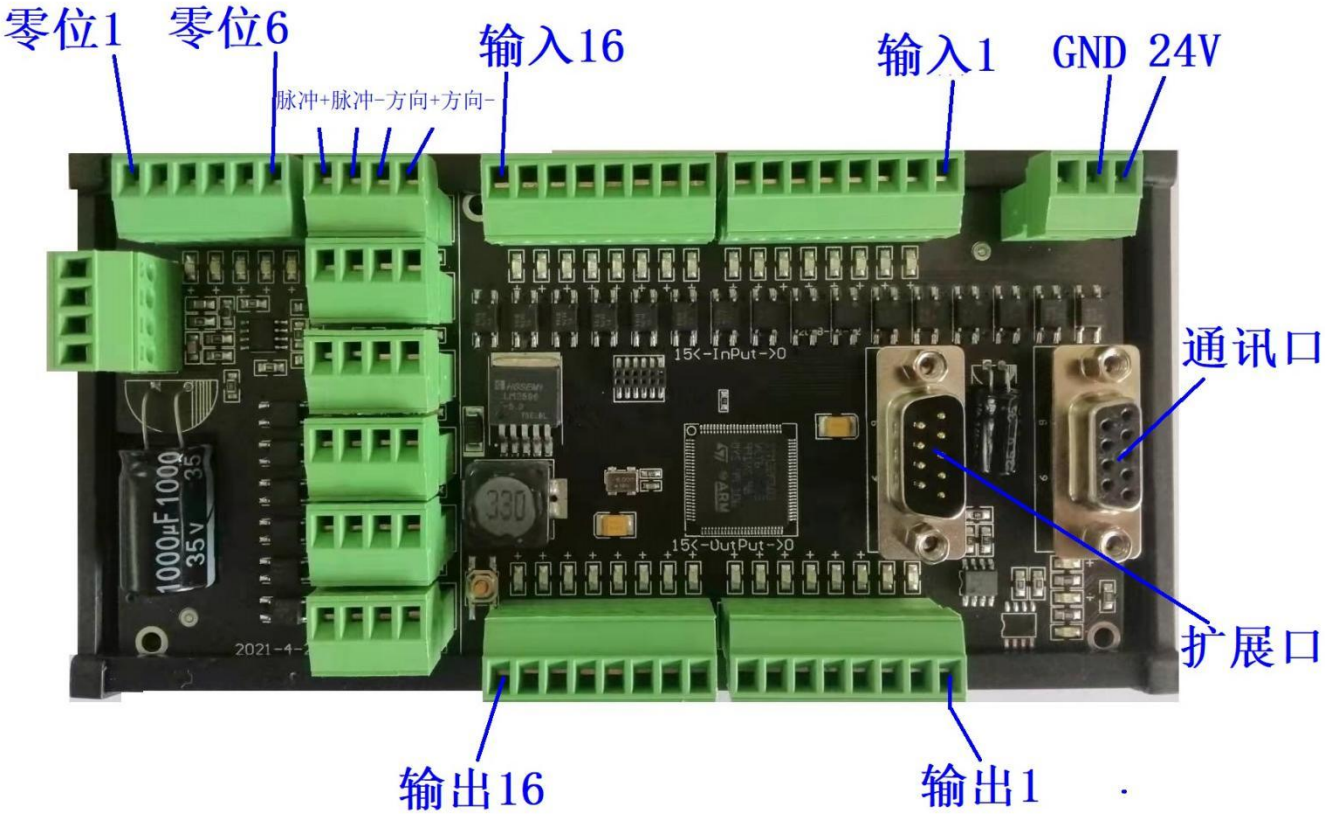
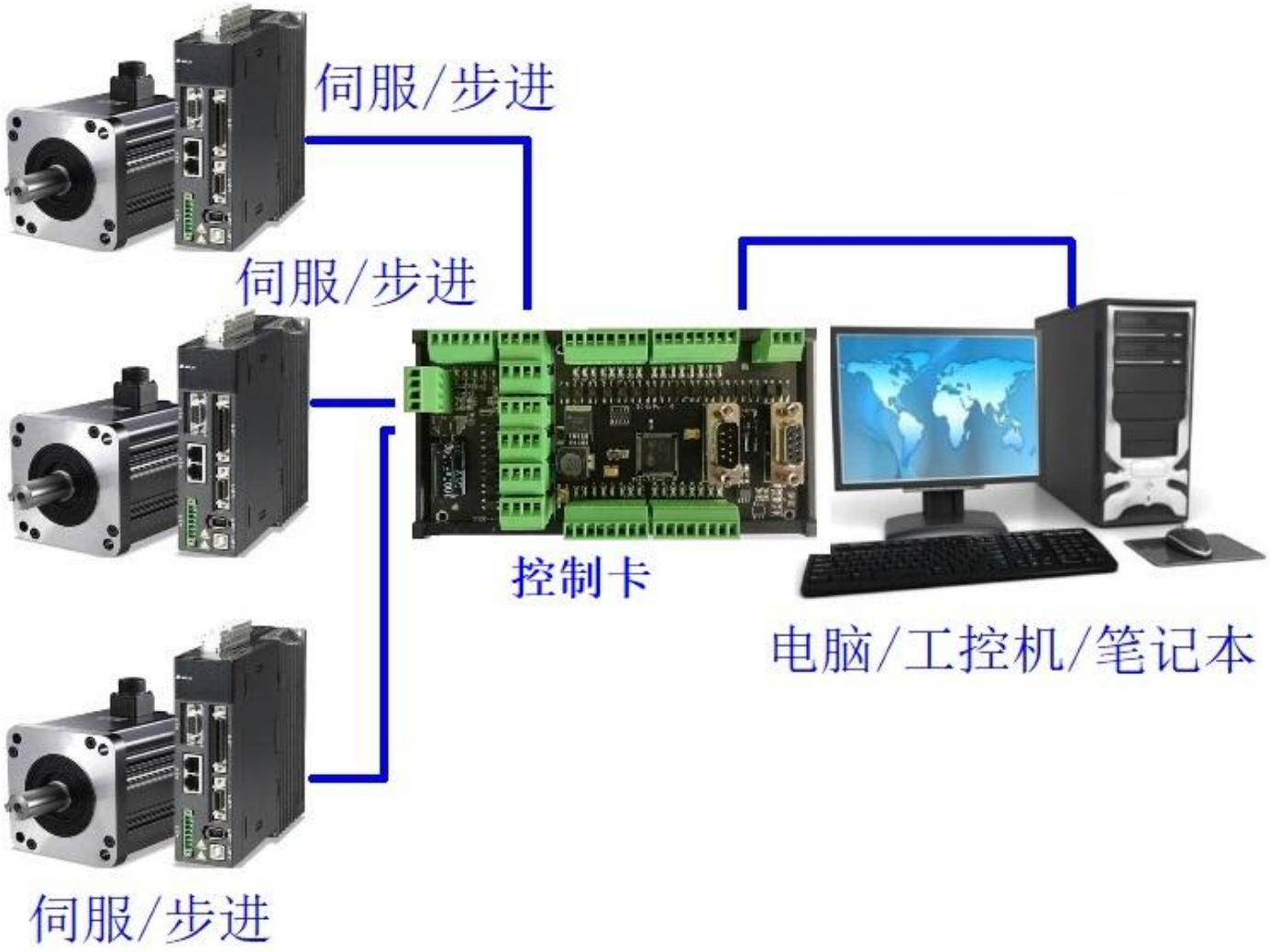
COM_GAS_N 运动控制卡用户手册 V2.9

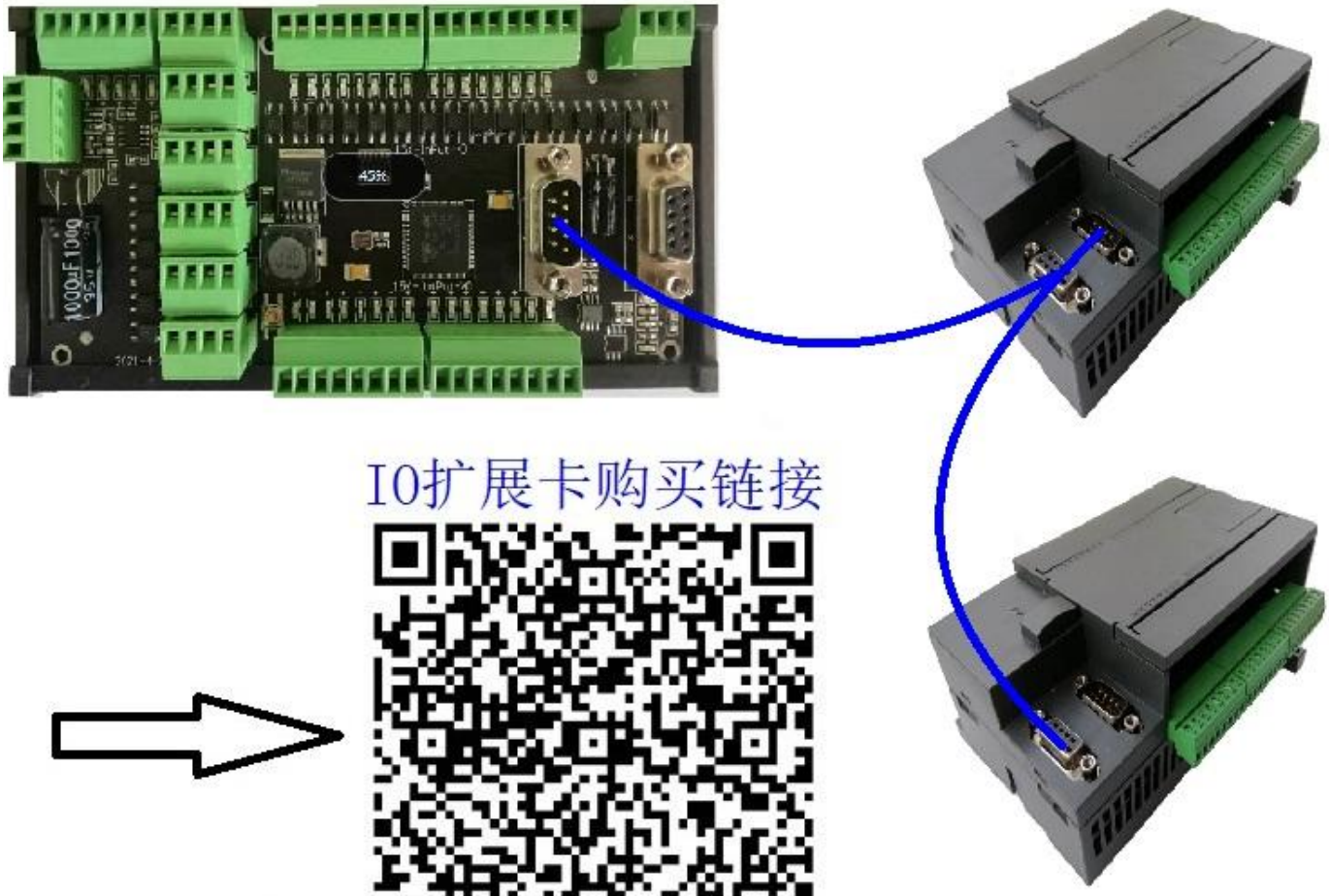
目录

一、硬件资源 5
二、软件资源 6
三、 硬件连接 7
3.1、通用输出 IO 接线方式 7
3.2、通用输入、零位输入接线方式 8
四、API 返回值及其意义 9
五、API 使用说明 10
5.1、板卡打开关闭 API 10
5.2、IO 操作 API 10
5.3、点位运动 API 12
5.4、JOG 运动 API 13
5.5、运动状态检测类 API 14
5.6、安全机制 API 15
5.7、其他指令 API 17
5.8、自动回零相关 API 19
六、 Demo 软件 DemoVC 21
七、IO 扩展方法 22
八、运动控制卡安装尺寸 23
九、附录-API 一览 24


零位1
零位6
脉冲+脉冲-方向+方向-
输入16
输入1
GND
24V
通讯口
扩展口
输出16
输出1
15<-InPut->0
15<-UutPut->0
1000μF 1000
35V
330


伺服/步进
伺服/步进
控制卡
电脑/工控机/笔记本
伺服/步进

支持IO级联扩展



IO扩展卡购买链接

支付宝扫一扫

一、硬件资源

1、控制卡自身有 16 路通用输入、16 路通用输出，输入采用光耦隔离，输出可直接驱动继电器。
2、控制卡支持 IO 扩展，可以扩展为 32、48、64、128、256、512……路输入输出。
3、控制卡有 6 路轴通道。每一路都包含脉冲、方向。
4、控制卡有 6 路零位输入。
5、控制卡支持正负硬限位，可以将任一零位输入或者通用 IO 输入配置映射为指定轴的硬限位信号。
6、板卡采用 24V 直流电源供电。

二、软件资源

控制卡提供了 VC 及 C#下的动态库，用户可利用动态库提供的 API 完成板卡打开、关闭、IO 输入输出、电机点位运动控制. Labview 下也可以使用。同时板卡支持 Linux、Android、iOS、Wince、Python、QT 等开发环境及语言。

名称	修改日期	类型	大小
测试软件-EthBoardTest_2016	2019/1/23 17:44	文件夹	
开发例程源代码（基于VS2010）	2019/1/23 17:44	文件夹	
库文件及头文件	2019/1/23 17:44	文件夹	
8轴运动控制卡用户手册V2.0.pdf	2018/7/1 14:46	Adobe Acrobat ...	1,663 KB
开发工具软件VS2010下载地址（百度云盘链接）.txt	2018/7/24 20:55	文本文档	1 KB

三、硬件连接

3.1、通用输出 IO 接线方式

注:千万不可将 24V 直接接入输出 IO，可能导致输出端口短路，烧坏板卡！！！

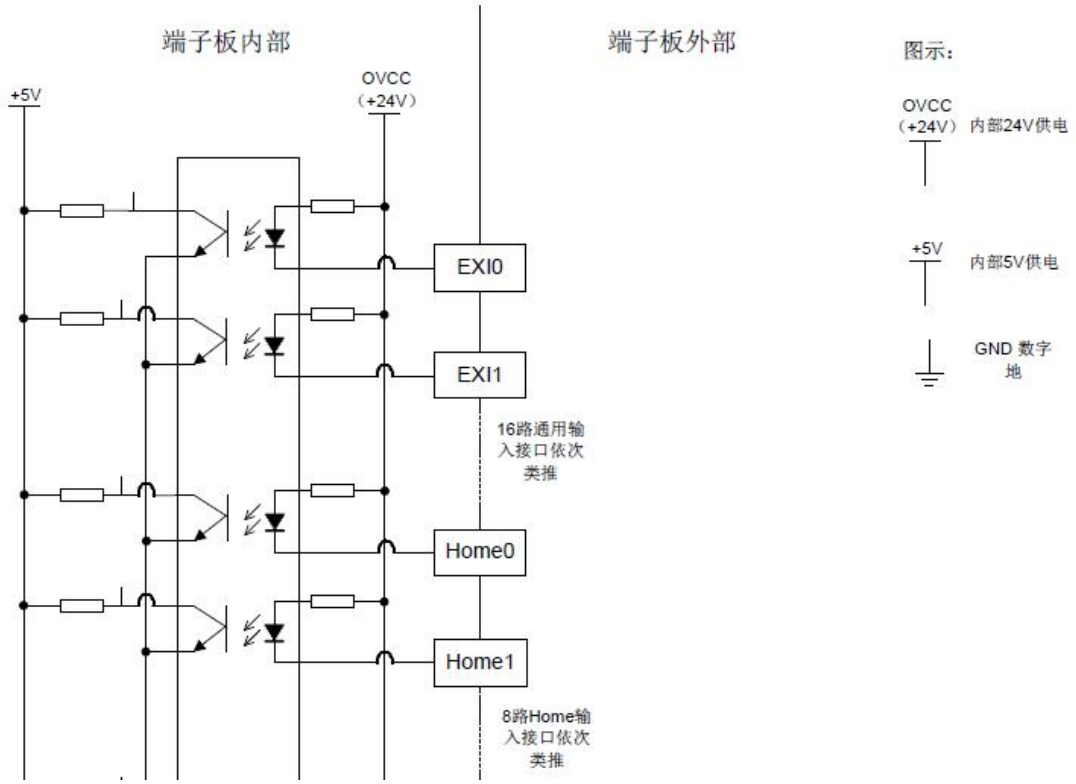
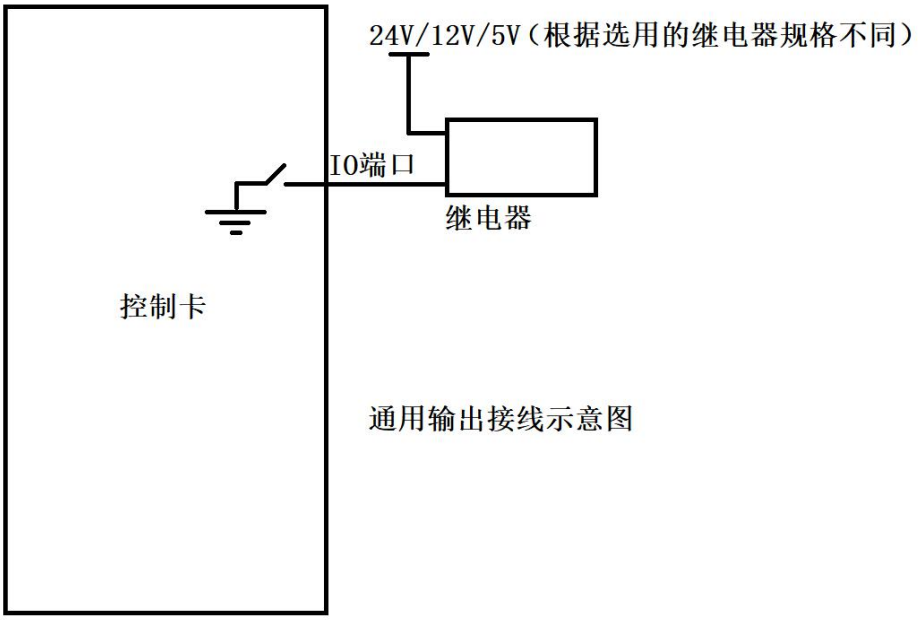


图 2-1：端子板通用输入信号内部电路示意图



通用输出接线示意图

3.2、通用输入、零位输入接线方式

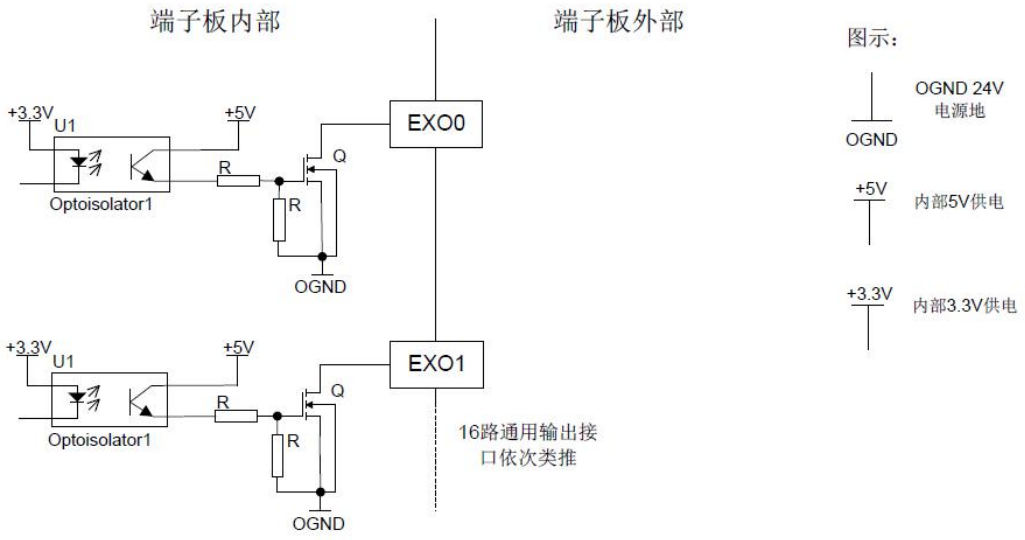
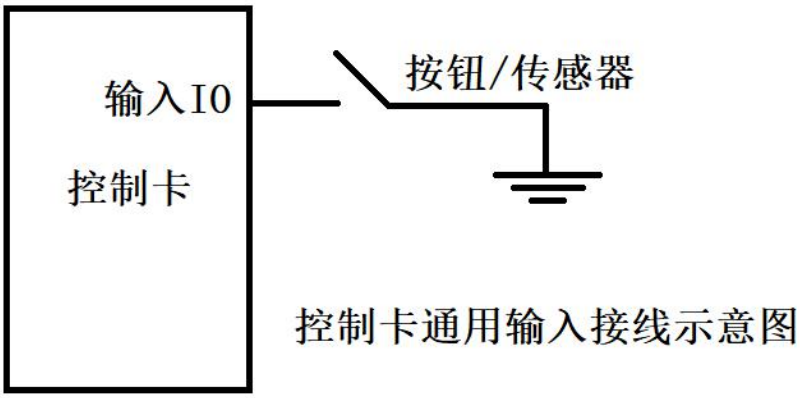


图 2-2：端子板通用数字输出信号内部电路示意图



四、API 返回值及其意义

返回值	意义	处理方法
0	执行成功	
1	执行失败	检测命令执行条件是否满足
2	版本不支持该 API	如有需要，联系厂家
7	参数错误	检测参数是否合理
-1	通讯失败	接线是否牢靠，更换板卡
-6	打开控制器失败	IP 配置是否正确，是否调用 2 次 GA_Open
-7	运动控制器无响应	检测运动控制器是否连接，是否打开。更换板卡

五、API 使用说明

5.1、板卡打开关闭 API

API	说明
GA_Open	打开板卡
GA_Reset	复位板卡
GA_Close	关闭板卡

参数详细说明：

int GA_Open(short iType=0,char* cName="COM1")	
iType	打开方式，0 网口，1 串口
cName	当 iType=0（网口方式打开）时，该参数代表 PC 端 IP 地址 当 iType=1（串口方式打开）时，该参数代表默认串口号 如果不知道确切串口号，可设置为空，系统自动搜索
int GA_Reset()	
无参数	
int GA_Close()	
无参数	

示例代码：

```
int iRes = 0;

//iRes += GA_Open(1,"");
iRes += GA_Open(1,"COM1");//通过串口打开板卡，串口号为 COM1(如果设置为空，系统自动搜索)
iRes += GA_Reset();//复位板卡
iRes += GA_Close();//关闭板卡
```

5.2、IO 操作 API

API	说明
GA_GetDiRaw	获取 IO 输入（包含主卡 IO、限位、零位）
GA_SetExtDoValue	设置 IO 输出（包含主模块和扩展模块）
GA_GetExtDiValue	获取 IO 输入（包含主模块和扩展模块）
GA_GetExtDoValue	获取 IO 输出（包含主模块和扩展模块）
GA_SetExtDoBit	设置指定 IO 模块的指定位输出（包含主模块和扩展模块）
GA_GetExtDiBit	获取指定 IO 模块的指定位输入（包含主模块和扩展模块）
GA_GetExtDoBit	获取指定 IO 模块的指定位输出（包含主模块和扩展模块）

参数详细说明：

int GA_GetDiRaw(short diType,long *pValue);	
diType	指定数字 IO 类型 MC_HOME(该宏定义为 3) 原点开关 MC_GPI(该宏定义为 4) 通用输入

pValue	IO 输入值存放指针
int GA_SetExtDoValue(short nCardIndex,unsigned long *value,short nCount=1)	
nCardIndex	起始板卡索引(0~63),0 是主模块，扩展模块从 1 开始
value	IO 输出值存放指针
nCount	本次设置的模块数量（1~64）
int GA_GetExtDiValue(short nCardIndex,unsigned long *pValue,short nCount=1)	
nCardIndex	起始板卡索引(0~63),0 是主模块，扩展模块从 1 开始
pValue	IO 输入值存放指针
nCount	本次获取的模块数量(1~64)
int GA_GetExtDoValue(short nCardIndex,unsigned long *pValue,short nCount=1)	
nCardIndex	起始板卡索引(0~63),0 是主模块，扩展模块从 1 开始
pValue	IO 输出值存放指针
nCount	本次获取的模块数量(1~64)
int GA_SetExtDoBit(short nCardIndex,short nBitIndex,unsigned short nValue)	
nCardIndex	起始板卡索引(0~63),0 是主模块，扩展模块从 1 开始
nBitIndex	IO 位索引号(0~15)
nValue	IO 输出值(0/1)
int GA_GetExtDiBit(short nCardIndex,short nBitIndex,unsigned short *pValue)	
nCardIndex	起始板卡索引(0~63),0 是主模块，扩展模块从 1 开始
nBitIndex	IO 位索引号(0~15)
pValue	IO 输入值存放指针
int GA_GetExtDoBit(short nCardIndex,short nBitIndex,unsigned short *pValue)	
nCardIndex	起始板卡索引(0~63),0 是主模块，扩展模块从 1 开始
nBitIndex	IO 位索引号(0~15)
pValue	IO 输入值存放指针

示例代码：

```
int iRes = 0;

//iRes += GA_Open(1,"");
iRes += GA_Open(1,"COM1");//通过串口打开板卡，串口号为 COM1(如果设置为空，系统自动搜索)
iRes += GA_Reset();//复位板卡
iRes += GA_SetExtDoBit(0,4,1);//设置板卡 1 端口 3IO 输出
iRes += GA_SetExtDoBit(0,4,0);//关闭板卡 1 端口 3IO 输出
iRes += GA_Close();//关闭板卡
```

5.3、点位运动 API

API	说明
GA_PrfTrap	设置指定轴为点位模式
GA_SetTrapPrm	设置点位模式运动参数
GA_GetTrapPrm	读取点位模式运动参数
GA_SetPos	设置目标位置
GA_SetVel	设置目标速度
GA_Update	启动点位运动

参数详细说明：

int GA_PrfTrap(short iAxis)	
iAxis	规划轴号
int GA_SetTrapPrm(short iAxis,TTrapPrm* pPrm)	
iAxis	规划轴号
pPrm	设置点位模式运动参数
int GA_GetTrapPrm(short iAxis,TTrapPrm* pPrm)	
iAxis	规划轴号
pPrm	读取点位模式运动参数
int GA_SetPos(short iAxis,long Pos)	
iAxis	规划轴号
pos	设置目标位置，单位是脉冲
int GA_SetVel(short iAxis,double Vel)	
iAxis	规划轴号
vel	设置目标速度，单位是“脉冲/毫秒”
int GA_Update(long mask)	
mask	按位指示需要启动点位运动的轴号 bit0 表示轴，bit1 表示 2 轴，……

示例代码：

```
int iRes = 0;
//iRes += GA_Open(1,"");
iRes += GA_Open(1,"COM1");//通过串口打开板卡，串口号为 COM1(如果设置为空，系统自动搜索)
iRes += GA_Reset();//复位板卡
iRes += GA_AxisOn(1);//设置轴 1 使能
iRes += GA_PrfTrap(1);//设置板卡轴 1 为点位模式
TTrapPrm TrapPrm;
TrapPrm.acc = 0.5;//设置点位运动加速度为 0.5 脉冲/毫秒^2
TrapPrm.dec = 0.5;//设置点位运动减速度为 0.5 脉冲/毫秒^2
TrapPrm.velStart = 0;//设置点位运动起始速度为 0 脉冲/毫秒
TrapPrm.smoothTime = 0;//设置点位运动平滑时间为 0
```

```
iRes += GA_SetTrapPrm(m_iAxisNum,&TrapPrm);
iRes += GA_SetVel(1,10);//设置轴 1 点位运动速度为 0 脉冲/毫秒
iRes += GA_SetPos(1,10000);//设置轴 1 目标位置为 10000
iRes += GA_Update(0XFF);//启动点位运动
```

5.4、JOG 运动 API

API	说明
GA_PrfJog	设置指定轴为 JOG 模式(速度模式)
GA_SetJogPrm	设置 JOG 模式运动参数
GA_GetJogPrm	读取 JOG 模式运动参数
GA_SetVel	设置目标速度
GA_Update	启动 JOG 运动

参数详细说明：

int GA_PrfJog(short iAxis)	
iAxis	iAxis
int GA_SetJogPrm(short iAxis,TJogPrm* pPrm)	
iAxis	iAxis
pPrm	设置 Jog 模式运动参数
int GA_GetJogPrm(short iAxis,TJogPrm* pPrm)	
iAxis	iAxis
pPrm	获取 Jog 模式运动参数
int GA_SetVel(short iAxis,double Vel)	
iAxis	规划轴号
vel	设置目标速度，单位是“脉冲/毫秒”
int GA_Update(long mask)	
mask	按位指示需要启动JOG运动的轴号 bit0 表示轴，bit1 表示 2 轴，……

示例代码：

```
int iRes = 0;

//iRes += GA_Open(1,"");
iRes += GA_Open(1,"COM1");//通过串口打开板卡，串口号为 COM1(如果设置为空，系统自动搜索)
iRes += GA_Reset();//复位板卡

iRes += GA_PrfJog(1);//设置轴 1 为 Jog 模式
iRes += GA_AxisOn(1);//设置轴 1 使能

TJogPrm JogPrm;
JogPrm.dSmooth = 0;
JogPrm.dAcc = 0.5;//设置 JOG 运动加速度为 0.5 脉冲/毫秒^2
JogPrm.dDec = 0.5;//设置点位运动加速度为 0.5 脉冲/毫秒^2
```

```
//注意：如果轴当前模式不是 Jog 模式，设置 JOG 运动参数会失败，会返回 1
iRes = GA_SetJogPrm(1,&JogPrm);//设置 JOG 运动参数
iRes = GA_SetVel(1,-1*fabs(50));//设置 JOG 运动速度为 50 脉冲/毫秒^2
iRes += GA_Update(0XFF);//启动 JOG 运动
```

5.5、运动状态检测类 API

API	说明
GA_Stop	停止一个或多个轴的规划运动，停止坐标系运动
GA_GetSts	读取轴状态
GA_ClrSts	清除驱动器报警标志、跟随误差越限标志、限位触发标志
GA_GetPrfPos	读取规划位置

参数详细说明：

int GA_Stop(long lMask,long lOption)	
lMask	按位指示需要停止运动的轴号或者坐标系号 bit0表示轴1，bit1表示轴2，„，bit7表示轴8 bit8表示坐标系1，bit9表示坐标系2 当 bit 位为 1 时表示停止对应的轴或者坐标系
lOption	按位指示停止方式 bit0表示轴1，bit1表示轴2，„，bit7表示轴8 bit8表示坐标系1，bit9表示坐标系2 当bit位为0时表示平滑停止对应的轴或坐标系 当bit位为1时表示紧急停止对应的轴或坐标系
int GA_GetSts(short nAxisNum,long *pSts,short nCount=1,unsignedlong *pClock=NULL)	
nAxisNum	起始轴号
pSts	32 位轴状态字，详细定义参见光盘头文件 GAS_N.h 的轴状态位定义部分
nCount	读取的轴数，默认为　1次最多可以读取个轴的状态
pClock	读取控制器时钟，默认为：NULL，即不用读取控制器时钟
int GA_ClrSts(short nAxisNum,short nCount)	
nAxisNum	起始轴号，取值范围：[1,AXIS_MAX_COUNT]
nCount	清除的轴数，默认为,1次最多可以清除个轴的异常状态
int GA_GetPrfPos(short nAxisNum,double *pValue,short nCount=1,unsignedlong *pClock=NULL)	
nAxisNum	起始轴号，取值范围：[1,AXIS_MAX_COUNT]
pValue	规划位置
nCount	读取的规划轴数，默认为,1次最多可以读取个轴的运动模式
pClock	读取控制器时钟，默认为：NULL，即不用读取控制器时钟

示例代码：

```
int iRes = 0;
long lSts = 0;
double dPrfPos = 0;
```

```
//iRes += GA_Open(1,"");
iRes += GA_Open(1,"COM1");//通过串口打开板卡，串口号为 COM1(如果设置为空，系统自动搜索)
iRes += GA_Reset();//复位板卡

iRes += GA_Stop((0X0001 << (5-1)),0);//设置轴 5 停止运动
iRes += GA_GetSts(1,&lSts);//获取轴 1 状态
iRes += GA_ClrSts(1);//清除轴 1 运动状态
iRes += GA_GetPrfPos(1,&dPrfPos ,1,NULL)
```

5.6、安全机制 API

重点说明：
1、默认各轴软限位及硬限位不起作用。
2、通过 GA_SetSoftLimit 函数可启用软限位。
3、通过 GA_LmtsOn 函数可启用硬限位。
4、零位或者通用输入都可以映射为轴的硬限位输入，设置函数为 GA_SetHardLimP 和 GA_SetHardLimN

API	说明
GA_SetSoftLimit	设置软限位
GA_GetSoftLimit	获取软限位
GA_SetHardLimP	设置正硬限位映射 IO
GA_SetHardLimN	设置负硬限位映射 IO
GA_LmtsOn	设置轴限位信号有效
GA_LmtsOff	设置轴限位信号无效
GA_LmtSns	设置运动控制器各轴限位触发电平

参数详细说明：

int GA_SetSoftLimit(short axis,long positive ,long negative)	
axis	轴编号
positive	正限位，单位脉冲
negative	负限位，单位脉冲
int GA_GetSoftLimit(short axis,long *pPositive ,long *pNegative)	
axis	轴编号
pPositive	正限位存放指针，单位脉冲
pNegative	负限位存放指针，单位脉冲
int GA_SetHardLimP(short nAxisNum,short nType ,short nCardIndex,short nIOIndex)	
nAxisNum	轴编号
nType	-1:硬限位无效，0：零位信号，1：通用输入 IO
nCardIndex	卡号，主卡为 0，扩展 IO 卡依次为 1、2、3、4.....
nIOIndex	IO 索引，0~15
int GA_SetHardLimN(short nAxisNum,short nType ,short nCardIndex,short nIOIndex)	
nAxisNum	轴编号
nType	-1:硬限位无效，0：零位信号，1：通用输入 IO
nCardIndex	卡号，主卡为 0，扩展 IO 卡依次为 1、2、3、4.....

nIOIndex	IO 索引，0~15
int GA_LmtsOn(short nAxisNum,short limitType = -1)	
nAxisNum	控制轴号，取值范围：[1,AXIS_MAX_COUNT]
limitType	需要有效的限位类型 MC_LIMIT_POSITIVE(该宏定义为0)：需要将该轴的正限位有效 MC_LIMIT_NEGATIVE(该宏定义为1)：需要将该轴的负限位有效 -1：需要将该轴的正限位和负限位都有效，默认为该值
int GA_LmtsOff(short nAxisNum,short limitType=-1)	
nAxisNum	控制轴号，取值范围：[1,AXIS_MAX_COUNT]
limitType	需要有效的限位类型 MC_LIMIT_POSITIVE(该宏定义为0)：需要将该轴的正限位无效 MC_LIMIT_NEGATIVE(该宏定义为1)：需要将该轴的负限位无效 -1：需要将该轴的正限位和负限位都无效，默认为该值
int GA_LmtSns(unsigned short nSense)	
nSense	按位标识轴的限位的触发电平状态，运动控制器默认的限位开关是常闭开关，即各轴处于正常工作状态时，其限位信号输入为低电平，当限位信号为高电平时，指令GA_LmtSns()的参数设置各轴正负限位开关的触发电平，当该参数的某个状态位时，表示将对应的限位开关设置为高电平触发，当某个状态位为时，表示将对应的限开关设置为低电平触发。 指令参数和各轴限位的对应关系如下所示： 状态位 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0 限位开关 轴 8 轴 7 轴 6 轴 5 轴 4 轴 3 轴 2 轴 1

示例代码：

```
int iRes = 0;
long lSts = 0;
long lSoftLimPos= 0X7FFFFFFF;
long lSoftLimNeg=0X80000000

//iRes += GA_Open(1,"");
iRes += GA_Open(1,"COM1");//通过串口打开板卡，串口号为 COM1(如果设置为空，系统自动搜索)
iRes += GA_Reset();//复位板卡
iRes = GA_SetSoftLimit(1,lSoftLimPos,lSoftLimNeg);

iRes = GA_LmtsOn(1,-1);//将第一个轴的正限位和负限位都有效
iRes = GA_SetHardLimN(1,1,0,0);//设置第一个轴的负限位为主卡通用输入 IO-0
```

5.7、其他指令 API

API	说明
GA_ZeroPos	清零轴的规划和编码器位置
GA_GetID	获取板卡唯一标识 ID
GA_SetBacklash	设置反向间隙补偿的相关参数
GA_GetBacklash	读取反向间隙补偿的相关参数

参数详细说明：

int GA_ZeroPos(short nAxisNum,short nCount=1)	
nAxisNum	需要位置清零的起始轴号,取值范围：[1,AXIS_MAX_COUNT]
nCount	需要位置清零的轴数
int GA_GetID(unsigned long* pID)	
pID	指向存放 ID 地址的指针
int GA_SetBacklash(short nAxisNum,long lCompValue,double dCompChangeValue,long lCompDir)	
nAxisNum	需要进行反向间隙补偿的轴的编号，取值范围：[1,8]
lCompValue	反向间隙补偿值，当为 0 时表示没有使能反向间隙补偿功能，取值只能为正，范围：[0, 1073741824]，单位：脉冲
dCompChangeValue	反向间隙补偿的变化量，取值只能为正，范围：[0, 1073741824]，单位：脉冲/毫秒 当该参数的值为 0 或者大于等于 lCompValue 时，则反向间隙的补偿量将瞬间叠加在规划位置上，没有渐变的过程
lCompDir	反向间隙补偿方向 0：只补偿负方向，当电机向负方向运动时，将施加补偿量，电机向正方向运动时，不施加补偿量 1：只补偿正方向，当电机向正方向运动时，将施加补偿量，电机向负方向运动时，不施加补偿量
int GA_GetBacklash(short nAxisNum,long *pCompValue,double *pCompChangeValue,long *pCompDir)	
nAxisNum	nAxisNum 查询的轴号，取值范围：[1,AXIS_MAX_COUNT]
pCompValue	pCompValue 读取的反向间隙补偿值

pCompChangeValue	pCompChangeValue　读取的反向间隙补偿值的变化量
pCompDir	pCompDir　读取的反向间隙补偿的补偿方向

示例代码：

```
int iRes = 0;
long lSts = 0;
unsigned long ulID = 0;

long lSoftLimPos= 0X7FFFFFFF;
long lSoftLimNeg=0X80000000

//iRes += GA_Open(1,"");
iRes += GA_Open(1,"COM1");//通过串口打开板卡，串口号为 COM1(如果设置为空，系统自动搜索)
iRes += GA_Reset();//复位板卡
iRes += GA_ZeroPos(1,8);//清零所有 8 个轴的位置

iRes = GA_GetID(&ulID);

//设置轴 1 反向间隙为 3 个脉冲，补偿速率 2 脉冲/ms，反向运动时补偿
iRes = GA_SetBacklash(1,3,2,0);
```

5.8、自动回零相关 API

API	说明
GA_HomeStart	启动轴回零
GA_HomeStop	停止轴回零
GA_HomeSetPrm	设置回零参数
GA_HomeSetPrmSingle	设置回零参数（非结构体方式）
GA_HomeGetPrm	获取回零参数
GA_HomeGetPrmSingle	获取回零参数（非结构体方式）
GA_HomeGetSts	获取回零状态

参数详细说明：

int GA_HomeStart(short iAxisNum)	
nAxisNum	需要回零的轴号,取值范围：[1,AXIS_MAX_COUNT]
int GA_HomeStop(short iAxisNum)	
nAxisNum	需要停止回零的轴号,取值范围：[1,AXIS_MAX_COUNT]
int GA_HomeSetPrm(short iAxisNum,TAxisHomePrm *pAxisHomePrm)	
nAxisNum	需要设置回零参数的轴号,取值范围：[1,AXIS_MAX_COUNT]

pAxisHomePrm	//轴回零参数
	typedef struct _AxisHomeParm{
	short nHomeMode; //1--HOME 回原点 2--HOME 加 Index 回原点
	short nHomeDir; //回零方向，1-正向回零，0-负向回零
	long lOffset; //回零偏移，回到零位后再走一个 Offset 作为零位
	double dHomeRapidVel; //回零快移速度，单位：Pluse/ms
	double dHomeLocatVel; //回零定位速度，单位：Pluse/ms
	double dHomeIndexVel; //回零寻找 INDEX 速度，单位：Pluse/ms
	double dHomeAcc; //回零使用的加速度
	}TAxisHomePrm;

int GA_HomeSetPrmSingle(short iAxisNum,short nHomeMode,short nHomeDir,long lOffset,double dHomeRapidVel,double	

dHomeLocatVel,double dHomeIndexVel,double dHomeAcc)	
nAxisNum	需要设置回零参数的轴号,取值范围：[1,AXIS_MAX_COUNT]
nHomeMode	1--HOME 回原点 2--HOME 加 Index 回原点
nHomeDir	回零方向，1-正向回零，0-负向回零
lOffset	回零偏移，回到零位后再走一个 Offset 作为零位
dHomeRapidVel	回零快移速度，单位：Pluse/ms
dHomeLocatVel	回零定位速度，单位：Pluse/ms
dHomeIndexVel	回零寻找 INDEX 速度，单位：Pluse/ms
dHomeAcc	回零使用的加速度
int GA_HomeGetPrm(short iAxisNum,TAxisHomePrm *pAxisHomePrm)	
nAxisNum	需要获取回零参数的轴号,取值范围：[1,AXIS_MAX_COUNT]

pAxisHomePrm	//轴回零参数
	typedef struct _AxisHomeParm{
	short nHomeMode; //1--HOME 回原点 2--HOME 加 Index 回原点
	short nHomeDir; //回零方向，1-正向回零，0-负向回零
	long lOffset; //回零偏移，回到零位后再走一个 Offset 作为零位
	double dHomeRapidVel; //回零快移速度，单位：Pluse/ms
	double dHomeLocatVel; //回零定位速度，单位：Pluse/ms
	double dHomeIndexVel; //回零寻找 INDEX 速度，单位：Pluse/ms
	double dHomeAcc; //回零使用的加速度
	}TAxisHomePrm;

int GA_HomeGetPrmSingle(short iAxisNum,short *nHomeMode,short *nHomeDir,long *lOffset,double* dHomeRapidVel,double* dHomeLocatVel,double* dHomeIndexVel,double* dHomeAcc)	
nAxisNum	需要获取回零参数的轴号,取值范围：[1,AXIS_MAX_COUNT]
nHomeMode	1--HOME 回原点 2--HOME 加 Index 回原点
nHomeDir	回零方向，1-正向回零，0-负向回零
lOffset	回零偏移，回到零位后再走一个 Offset 作为零位
dHomeRapidVel	回零快移速度，单位：Pluse/ms
dHomeLocatVel	回零定位速度，单位：Pluse/ms
dHomeIndexVel	回零寻找 INDEX 速度，单位：Pluse/ms
dHomeAcc	回零使用的加速度
int GA_HomeGetSts(short iAxisNum,unsigned short* pStatus)	
nAxisNum	需要获取回零状态的轴号,取值范围：[1,AXIS_MAX_COUNT]
pStatus	pStatus：指向回零状态的指针 0：尚未回零 1：回零中 2：回零成功

示例代码：

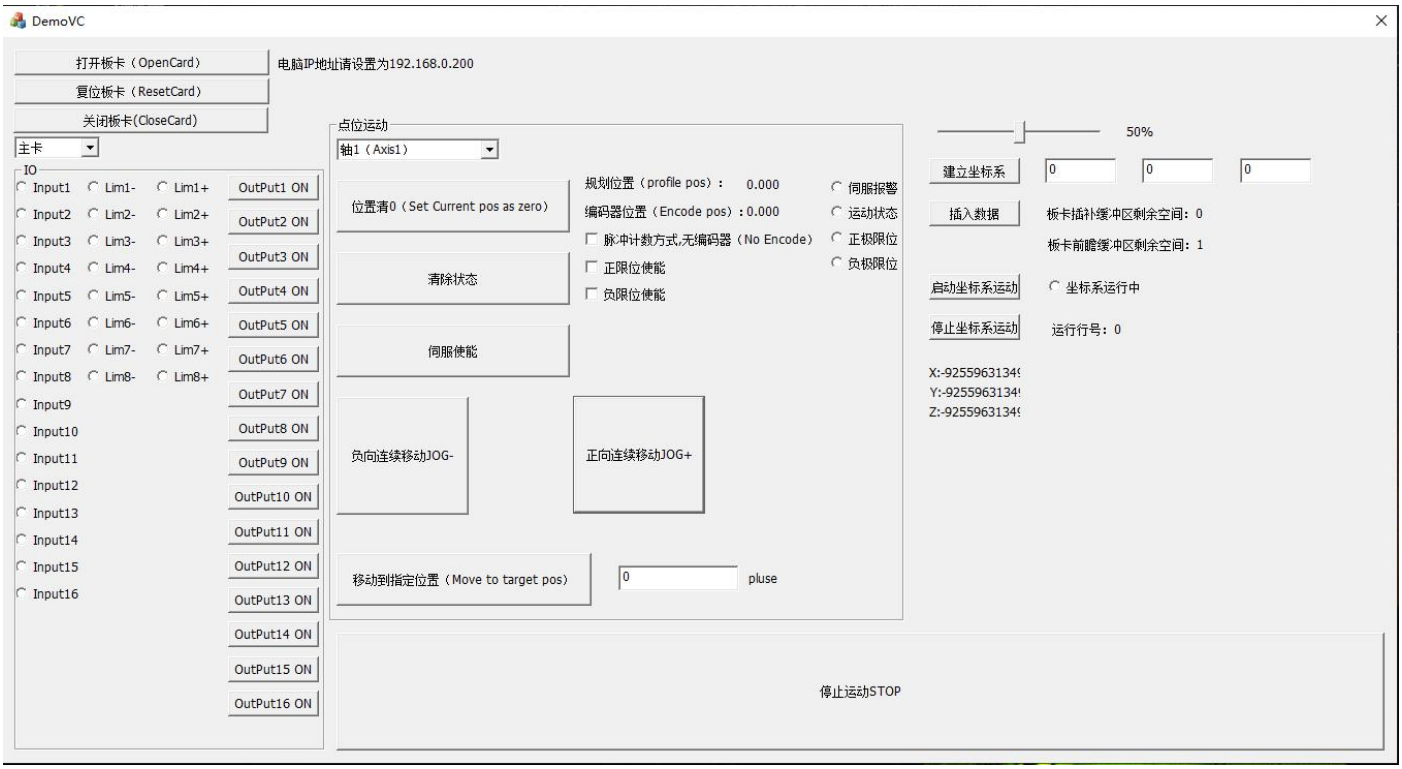
```
int iRes = 0;
short nStatus = 0;

TAxisHomePrm AxisHomePrm;
```

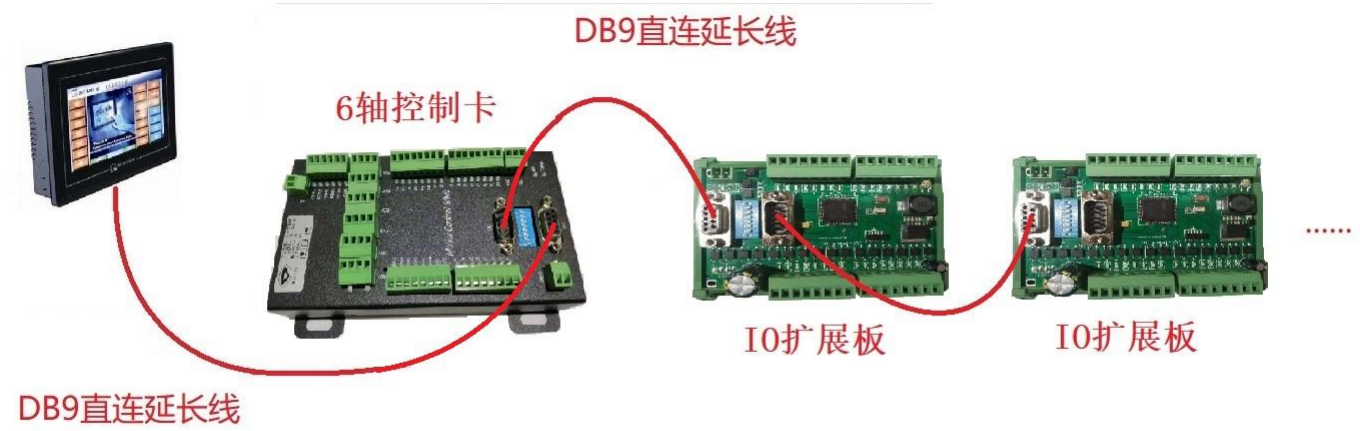
```
AxisHomePrm.nHomeMode = 1;
AxisHomePrm.nHomeDir = 0;
AxisHomePrm.dHomeRapidVel = 5;
AxisHomePrm.dHomeLocatVel = 1;
AxisHomePrm.dHomeIndexVel = 1;
AxisHomePrm.dHomeAcc = 0.2;

AxisHomePrm.lOffset = 0;
//设置轴 1 回零参数
iRes = GA_HomeSetPrm(1,&AxisHomePrm);
//启动轴 1 回零
iRes = GA_HomeStart(1);
//获取轴 1 回零状态
iRes = GA_HomeGetSts(1,&nStatus);
```

六、Demo 软件 DemoVC

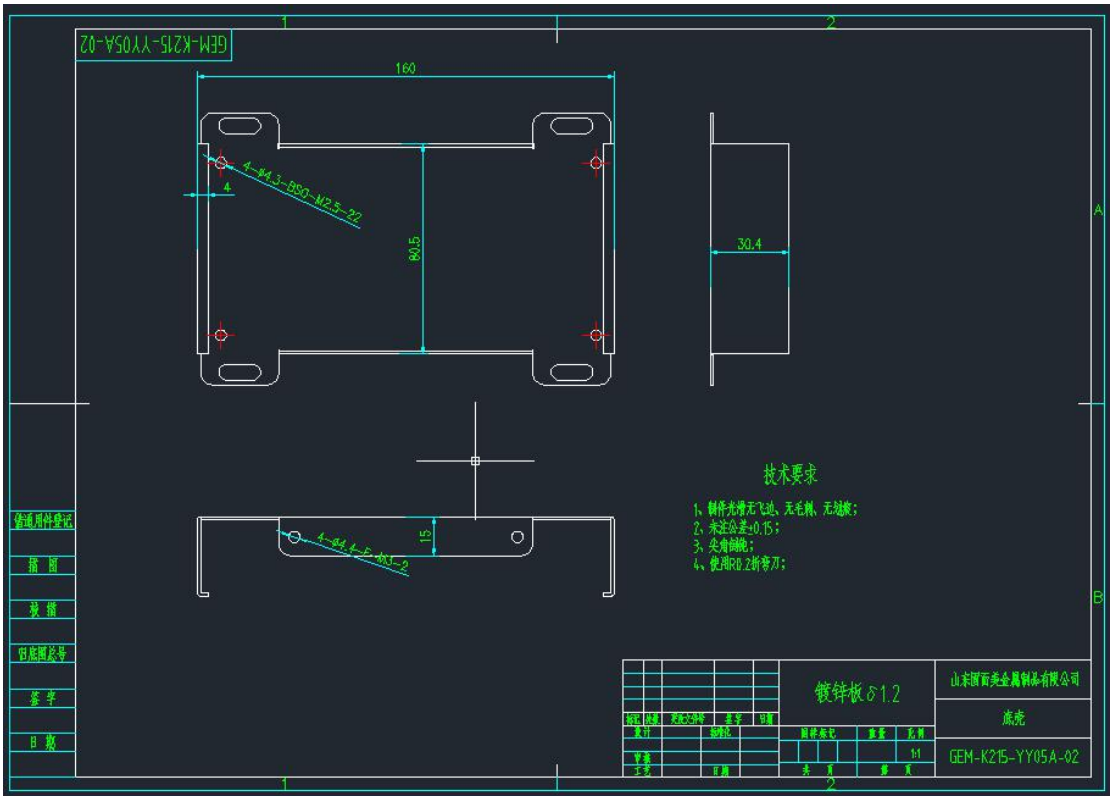


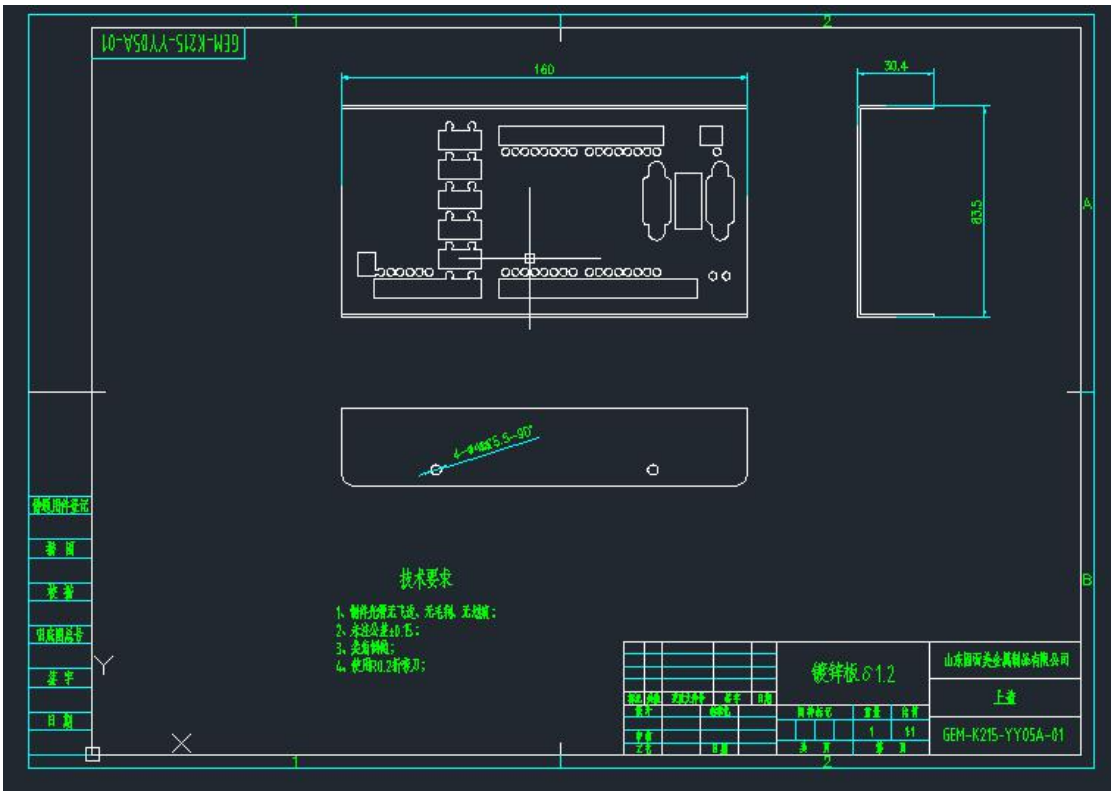
七、IO 扩展方法



八、运动控制卡安装尺寸

外形尺寸（长*宽*高 mm）(不含侧耳)	160	83.5	31.8
喷塑颜色	黑纱纹 SEP01128		
板厚（mm）	1.2		
其他事项	无		


GEM-K215-YY05A-02
160
30.4
60.5
技术要求
镀锌板δ1.2
山东国而美金属制品有限公司
底壳
GEM-K215-YY05A-02


GEM-K215-YY05A-01
160
30.4
技术要求
镀锌板δ1.2
山东国而美金属制品有限公司
上盖
GEM-K215-YY05A-01

九、附录-API 一览

板卡打开关闭 API	
GA_Open	打开板卡
GA_Reset	复位板卡
GA_Close	关闭板卡
IO 操作 API	
GA_GetDiRaw	获取 IO 输入（包含主卡 IO、限位、零位）
GA_SetExtDoValue	设置 IO 输出（包含主模块和扩展模块）
GA_GetExtDiValue	获取 IO 输入（包含主模块和扩展模块）
GA_GetExtDoValue	获取 IO 输出（包含主模块和扩展模块）
GA_SetExtDoBit	设置指定 IO 模块的指定位输出（包含主模块和扩展模块）
GA_GetExtDiBit	获取指定 IO 模块的指定位输入（包含主模块和扩展模块）
GA_GetExtDoBit	获取指定 IO 模块的指定位输出（包含主模块和扩展模块）
轴点位运动 API	
GA_PrfTrap	设置指定轴为点位模式
GA_SetTrapPrm	设置点位模式运动参数
GA_GetTrapPrm	读取点位模式运动参数
GA_SetPos	设置目标位置
GA_SetVel	设置目标速度
GA_Update	启动点位运动
轴 JOG 运动 API	
GA_PrfJog	设置指定轴为 JOG 模式(速度模式)
GA_SetJogPrm	设置 JOG 模式运动参数
GA_GetJogPrm	读取 JOG 模式运动参数
GA_SetVel	设置目标速度
GA_Update	启动 JOG 运动
运动状态检测类 API	
GA_Stop	停止一个或多个轴的规划运动，停止坐标系运动
GA_GetSts	读取轴状态
GA_ClrSts	清除驱动器报警标志、跟随误差越限标志、限位触发标志
GA_GetPrfPos	读取规划位置
安全机制 API	
GA_SetSoftLimit	设置软限位
GA_GetSoftLimit	获取软限位
GA_SetHardLimP	设置正硬限位映射 IO
GA_SetHardLimN	设置负硬限位映射 IO
GA_LmtsOn	设置轴限位信号有效
GA_LmtsOff	设置轴限位信号无效
GA_LmtSns	设置运动控制器各轴限位触发电平
其他 API	
GA_ZeroPos	清零轴的规划位置
GA_GetID	获取板卡唯一标识 ID
自动回零类 API	
GA_HomeStart	启动轴回零

GA_HomeStop	停止轴回零
GA_HomeSetPrm	设置回零参数
GA_HomeSetPrmSingle	设置回零参数（非结构体方式）
GA_HomeGetPrm	获取回零参数
GA_HomeGetPrmSingle	获取回零参数（非结构体方式）
GA_HomeGetSts	获取回零状态